Yaqeen Marketplace

Task: Connect React forms to a XAMPP (PHP + MySQL) backend using axios — the front-end↔back-end *integration*. Text forms only (image-upload forms skipped).

Stack: React 19 + Vite + react-hook-form + zod (front) | Apache + PHP + MySQL via XAMPP (back) | axios (bridge)

> **Result:** Registration, Contact, Admin Add User and Admin Edit User forms now save / update their data in the MySQL database `yaqeen_web_project` .

0. The Big Picture — how the pieces talk

```
React form (browser, http://localhost:5173)
   │   user fills form → react-hook-form + zod validate
   ▼   axios.post( url, data ) — sends JSON
PHP script (Apache, http://localhost, port 80)
   │   reads JSON body → builds SQL → runs query
   ▼
MySQL database "yaqeen_web_project" (table users / contacts)
   │   returns { success, message }
   ▼
React shows the message under the form
```

1. Start the XAMPP servers

Open **XAMPP Control Panel** and press **Start** on:

- **Apache** → runs PHP (ports 80, 443).
- **MySQL** → the database (port 3306).

> **Problem we hit:** MySQL would not start — `Port 3306 in use by "Unable to open process"` .
> **Cause:** a separate **MySQL80** Windows service was already using port 3306.
> **Fix:** Windows *Services* → **MySQL80** → **Stop** (optionally set Startup type = Manual). Then XAMPP MySQL started (turned green).

2. Create the database & first table (phpMyAdmin)

Open `http://localhost/phpmyadmin` . A database `yaqeen_web_project` already existed; we added the `users` table. SQL tab → paste → **Go**:

```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    account_type      VARCHAR(20),
    full_name         VARCHAR(100),
    email             VARCHAR(100),
    phone             VARCHAR(20),
    id_card           VARCHAR(20),
    city              VARCHAR(50),
    postal_code       VARCHAR(10),
    address           TEXT,
    business_name     VARCHAR(100),
    business_category VARCHAR(50),
    user_password     VARCHAR(100)
);
```

3. Install axios in the React project

In the project folder terminal:

```
npm install axios
```

This is the library that lets React send HTTP POST requests to PHP.

4. Backend folder + connection file

Created folder `D:\xampp\htdocs\yaqeen-backend\` (everything in `htdocs` is served by Apache).

connection.php — opens the MySQL connection

```php
<?php
    // XAMPP defaults: host=localhost, user=root, password="" (empty).
    $connect = mysqli_connect("localhost", "root", "", "yaqeen_web_project");

    if (!$connect) {
        die("Connection failed: " . mysqli_connect_error());
    }
?>
```

5. The registration backend (sir's pattern)

register_processing.php

```php
<?php
    // CORS headers: let the React app (different port) call this script.
    header("Access-Control-Allow-Origin: *");
    header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
    header("Access-Control-Allow-Headers: Content-Type");

    // Browser sends a preflight OPTIONS request first — answer OK and stop.
    if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
        http_response_code(200);
        exit();
    }

    include("connection.php");

    // axios sends JSON in the body — read it and decode to a PHP object.
    $rawData = file_get_contents("php://input");
    $data = json_decode($rawData);

    if (isset($data->email) && isset($data->password)) {
        $accountType = $data->accountType;
        $fullName    = $data->fullName;
        $email       = $data->email;
        // ...the other fields...
        $password    = $data->password;

        $query = "INSERT INTO users
                  (account_type, full_name, email, phone, id_card, city,
                   postal_code, address, business_name, business_category, user_password)
                  VALUES
                  ('$accountType','$fullName','$email','$phone','$idCard','$city',
                   '$postalCode','$address','$businessName','$businessCategory','$password')";
```

```
        $process_query = mysqli_query($connect, $query);

        if ($process_query) {
            echo json_encode(["success"=>true, "message"=>"Account created successfully for
$fullName."]);
        } else {
            echo json_encode(["success"=>false, "message"=>"Database insertion failed:
".mysqli_error($connect)]);
        }
    } else {
        echo json_encode(["success"=>false, "message"=>"No data was received by the
server."]);
    }
?>
```

6. The axios service file (React side)

Created `src/serviceApi.js` — one function per form. Each posts to its PHP file and returns the response.

```
import axios from "axios";

// Apache here runs on port 80, so the URL is just localhost (no :8080).
export const process_registration = async (data) => {
  const response = await axios.post(
    "http://localhost/yaqeen-backend/register_processing.php", data);
  return response.data;
};

export const process_contact      = async (data) => { /* contact_processing.php */ };
export const process_user_add     = async (data) => { /* user_add_processing.php */ };
export const process_user_update  = async (data) => { /* user_update_processing.php */ };
```

7. Wire the Registration form

In `src/components/Registration.jsx` we changed 3 things:

a) imports + state

```
import { useState } from 'react';
import { process_registration } from '../serviceApi';
// inside component:
const [serverMessage, setServerMessage] = useState('');
```

b) submit becomes async & calls the API

```
async function submit(data) {
  // ...existing validation alerts stay...
  try {
    const result = await process_registration(data);
    setServerMessage(result.message);
  } catch (error) {
    setServerMessage('A server error 500 occurred.');
  }
}
```

c) show the message under the form

```
{serverMessage && (
  <p className="text-center mt-3 mb-0"><strong>{serverMessage}</strong></p>
)}
```

> **Bug we fixed:** the "I agree to terms" checkbox was registered but **not in the zod schema**. zod *strips* any field not declared in the schema, so `data.agreeTerms` was always `undefined` → the check failed even when ticked. Fix = add it to the schema:
>
> ```
> agreeTerms: z.boolean().optional(),
> ```

8. The other three forms (same recipe)

Each got the exact same 3-step treatment (import + state, async submit, show message), plus its own PHP file.

Form (file)	axios fn	PHP file	SQL action
Registration.jsx	process_registration	register_processing.php	INSERT users

Contact.jsx	process_contact	contact_processing.php	INSERT contacts
admin/UserAdd.jsx	process_user_add	user_add_processing.php	INSERT users
admin/UserEdit.jsx	process_user_update	user_update_processing.php	UPDATE users WHERE id

Edit User also sends the row `id` (taken from the URL) so PHP knows which row to update:

```
const result = await process_user_update({ ...data, id });
```

and the PHP uses it: `UPDATE users SET ... WHERE id = '$id'`. The password is only overwritten if a new one is typed (the placeholder `********` is ignored).

9. Extra tables for the new forms

Run once in phpMyAdmin (database `yaqeen_web_project`):

```
-- 1. Contact messages
CREATE TABLE contacts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100), email VARCHAR(100), phone VARCHAR(20),
    subject VARCHAR(150), message_type VARCHAR(30), priority VARCHAR(20),
    company VARCHAR(100), message TEXT
);

-- 2. Extra columns the admin user forms need
ALTER TABLE users
    ADD COLUMN role VARCHAR(20),
    ADD COLUMN status VARCHAR(20),
    ADD COLUMN join_date VARCHAR(20);
```

10. What was SKIPPED (on purpose)

Form	Why skipped
Product Add / Product Edit	They upload an **image** — sir said don't connect image-saving forms.
Login	Needs to *read* & check the DB (SELECT), not save. Not chosen this round.
Requests	Just a table with Approve/Reject buttons — no form data to save.

11. How to run it next time

#	Action
1	XAMPP Control Panel → Start **Apache** + **MySQL** (stop MySQL80 first if 3306 is busy).
2	In project folder: `npm run dev` → opens `http://localhost:5173`.
3	Fill any wired form → submit → green message → check the table in phpMyAdmin.

12. Files created / changed

Location	File	New / Changed
D:\xampp\htdocs\yaqeen-backend	connection.php	new
D:\xampp\htdocs\yaqeen-backend	register_processing.php	new
D:\xampp\htdocs\yaqeen-backend	contact_processing.php	new
D:\xampp\htdocs\yaqeen-backend	user_add_processing.php	new
D:\xampp\htdocs\yaqeen-backend	user_update_processing.php	new
src/	serviceApi.js	new (4 axios functions)
src/components/	Registration.jsx	changed (state + async submit + message + schema fix)
src/pages/	Contact.jsx	changed

src/pages/admin/	UserAdd.jsx	changed
src/pages/admin/	UserEdit.jsx	changed (also sends id)

> **Two things to know (not bugs):**
> 1. **Plain-text passwords** & SQL built by sticking text straight into the query — same as sir's classroom example. Fine for local learning; real apps use hashing + prepared statements (beyond current level).
> 2. **Duplicates allowed** — clicking submit twice inserts two rows. No "unique email" rule yet (that's a later topic).
